



Adding a new automation.

STANDARD OPERATING PROCEDURE — AUTOMATION PLATFORM

TYPE

01

SOP

DOCUMENT ID	SOP-AUTO-001
DEPARTMENT	IT / Automation Platform
OWNER	David
BACKUP	Juan
AUDIENCE	All DSS team members + AI assistants
VERSION	2.1
EFFECTIVE	May 2026

Purpose

This SOP defines how to add a new automation to the DSS Automation Platform. Read it end-to-end before generating any files. The platform supports three trigger types — file drops, scheduled runs, and incoming emails — and is authored by both developers and team members working with AI assistants. The default input pattern for any user-facing automation is email + Power Automate + SharePoint inbox folder, covered in section 6. Following this template ensures every automation looks the same, regardless of who built it.

Scope

Applies to all automations running on the DSS Mac mini under user 'claude'. Authors include David, Juan, and any DSS team member building an automation with assistance from Claude Desktop or another AI assistant. Out of scope: cloud-hosted automations, third-party SaaS workflows, and any code that does not live in the automation-platform repository.

TYPE

01

SOP

1. Platform overview

The DSS Automation Platform runs on a single Mac mini logged in as user 'claude', executing multiple automations from one git repo. All automations share the same infrastructure layer.

- Secret management via macOS Keychain (with the dss- prefix).
- Teams notifications via the notify-teams helper.
- Triggers: launchd (file drops, scheduled) plus the email-watcher daemon (incoming emails).
- Data storage via SharePoint Data Hub, OneDrive-synced to the mini.
- Status visibility via the DSS Automation > Status Teams channel.
- Files awaiting human approval routed to the DSS Automation > Review Teams channel.

Reference paths and repository:

```
Code:    /Users/claude/automation-platform/  
Data:    ~/dss-hub/          (symlinks into OneDrive-synced SharePoint)  
GitHub:  https://github.com/DSS-Distro/automation-platform
```

2. Who can author automations

Any DSS team member can propose and draft a new automation. The platform supports two authoring modes:

1. Direct authoring — David or Juan write code into the repo and deploy.
2. AI-assisted authoring — a team member describes the automation to Claude Desktop, which generates files following this SOP. Output is submitted to the review queue and deployed by David or Juan.

All AI-generated automations must pass review before launchd loads them. This SOP is the source of truth: when using Claude Desktop, attach this PDF (or paste its contents) at the start of the conversation as context. Section 13 covers the end-to-end Claude Desktop authoring flow.

TYPE

01

SOP

3. File structure for a new automation

Every automation is a self-contained folder under automations/ with the following structure. The manifest.yml registration card is required; everything else is optional but follows the conventions below.

```

automations/<your-automation-name>/
  manifest.yml          <- registration card (required)
  bin/                  <- shell scripts
    run.sh
    ...
  prompts/              <- Claude prompts as markdown (if used)
    *.md
  launchd/              <- one plist per trigger (file/scheduled only)
    com.dss.automation.<name>.plist
  README.md             <- what + why

```

4. Required: manifest.yml

Every automation must include a manifest.yml at its root. This is the registration card the platform uses to track ownership, criticality, dependencies, and trigger configuration.

```

name: <automation-name>
description: <one sentence>
owner: <email>
backup_owner: <name or email>
criticality: low | medium | high
trigger:
  type: file_drop | scheduled | email
  # one of:
  watch_path: <absolute path>          # if file_drop
  cron: "0 8 * * 2"                    # if scheduled
  subject_prefix: "SOP:"                # if email
  from_filter: "@digitalsmokesupplies.com" # if email
secrets:
  - <keychain-entry-name-without-dss-prefix>
inputs:
  - <path>
outputs:
  - <path>
review_required: true | false          # routes outputs to Review channel

```

TYPE

01

SOP

5. Trigger types

The platform supports three trigger types. Pick exactly one per automation. The trigger type determines how launchd or the email-watcher invokes the script and what arguments it receives.

5.1 File-drop trigger

- Fires when a file lands in a watched OneDrive folder.
- Implemented via launchd WatchPaths. See section 9.
- Use the REAL OneDrive path, not the symlinked ~/dss-hub/ path.
- Example use: meeting transcripts dropped into Inbox, vendor reports auto-imported.

5.2 Scheduled trigger

- Fires at a calendar time (daily, weekly, monthly).
- Implemented via launchd StartCalendarInterval. See section 9.
- Example use: weekly inventory pull every Tuesday 8am, end-of-day sales summary.

5.3 Email trigger

- Fires when an email arrives at the service mailbox with a recognized subject prefix.
- Two implementation patterns. Pattern A (RECOMMENDED, see section 6): Power Automate watches Outlook, drops attachments into a SharePoint inbox folder, the mini's file_drop watcher picks them up. Pattern B (advanced, see section 10): the email-watcher Python daemon polls IMAP directly on the mini.
- Service mailbox: claudio@digitalsmokesupplies.com
- Example use: meeting transcripts, SOP requests, customer email forwarding, report requests.

TYPE

01

SOP

5. TRIGGER TYPES (CONT.)

Reserved subject prefixes (case-insensitive). Pick one when registering an email-triggered automation:

SOP :	Generate or update an SOP. Body contains the description.
AUTOMATION :	Submit a new automation for review. Body or attachment contains the spec.
MEETING :	Process a meeting transcript. Attachment expected.
REPORT :	Generate a report. Body contains parameters.
FORWARD :	Forwarded customer email. Body contains the original message.

New prefixes must be added to this SOP and to the email-watcher router config before the automation is deployed. The email-watcher reads each automation's manifest.yml subject_prefix field to dispatch incoming mail.

Email triggers must verify sender.

The from_filter regex in manifest.yml is enforced by the email-watcher before the handler runs. Without a valid filter, anyone could trigger your automation. The minimum acceptable filter is "@digitalsmokesupplies.com".

TYPE

01

SOP

6. Power Automate input bridge (RECOMMENDED for email triggers)

This is the standard pattern for any automation that needs to receive email as input. Microsoft Power Automate watches the service mailbox, saves attachments to a SharePoint inbox folder, and the OneDrive sync brings the file to the mini where a file_drop trigger picks it up. This pattern is recommended over direct IMAP polling because it runs entirely in Microsoft 365 (no daemon to maintain), produces a visible audit trail in SharePoint, and works even when the mini is offline.

End-to-end flow:

```
User sends email
→ claud@digitalismokesupplies.com
→ Power Automate flow (subject filter)
→ SharePoint /06_AI_Workspace/Inbox/<automation>/
→ OneDrive sync to Mac mini
→ launchd file_drop trigger
→ automation runs
```

Inbox folder convention. Each email-triggered automation owns one subfolder under the SharePoint inbox, named exactly like the automation:

```
Shared Documents/06_AI_Workspace/Inbox/Meeting-automation/sales/
Shared Documents/06_AI_Workspace/Inbox/Meeting-automation/ops/
Shared Documents/06_AI_Workspace/Inbox/Sop-generator/
Shared Documents/06_AI_Workspace/Inbox/Report-requests/
```

The folder must exist in SharePoint before the Power Automate flow runs. Power Automate will not auto-create missing parent folders.

TYPE

01

SOP

6. POWER AUTOMATE INPUT BRIDGE (CONT.)

Standard Power Automate flow recipe. Use this exact recipe for every email-triggered automation — only the subject prefix and destination folder change.

Before you start:

- Confirm the destination folder exists in SharePoint.
- Note the SharePoint Site URL (the part up to and including the site name, e.g. <https://dtoxdss.sharepoint.com/sites/datahub>).
- Sign into make.powerautomate.com as claudio@digitalsmokesupplies.com.

Step 1 — Create the flow.

1. Click + Create → Automated cloud flow.
2. Name it: <Automation> attachment to SharePoint.
3. Trigger: "When a new email arrives (V3)" (Office 365 Outlook).
4. Click Create.

Step 2 — Configure the trigger. Click Show advanced options:

Folder:	Inbox
To:	claudio@digitalsmokesupplies.com
Subject Filter:	<YOUR PREFIX>: e.g. SALES MEETING:
Importance:	Any
Only with Attachments:	Yes
Include Attachments:	Yes

TYPE

01

SOP

6. POWER AUTOMATE INPUT BRIDGE (CONT.)**Step 3 — Add the loop.**

1. Click + New step.
2. Choose Apply to each (Control connector).
3. Input field: open Dynamic Content panel → click Attachments.

Step 4 — Add the SharePoint write action INSIDE the loop. Use Send an HTTP request to SharePoint, not Create file (Create file fails on overwrite by default).

```
Site Address: https://<tenant>.sharepoint.com/sites/<site>
Method:      POST
Uri:         _api/web/GetFolderByServerRelativeUrl(
              '/sites/<site>/Shared Documents/...path.../<automation>'
            )/Files/add(
              url='@{items('Apply_to_each')}['Name']}',
              overwrite=true
            )
Headers:
  Accept:      application/json;odata=verbose
  Content-Type: application/octet-stream
Body:         Attachments Content (from dynamic content, in loop)
```

Server-relative path must match Site Address.

The path inside `GetFolderByServerRelativeUrl(...)` must start with the same `/sites/<site>/` prefix as the Site Address. To get the exact path: open the destination folder in SharePoint, copy the URL bar, find the `id=` parameter, and URL-decode it (`%2F` → `/`, `%20` → space, `%5F` → `_`).

TYPE

01

SOP

6. POWER AUTOMATE INPUT BRIDGE (CONT.)**Step 5 — Test.**

1. Top-right → Save.
2. Top-right → Test → Manually → Test.
3. Send yourself an email to claude@digitalsmokesupplies.com with the matching subject prefix and a small attachment.
4. Within ~30 seconds, the run history shows the steps progressing.
5. Confirm the file appears in the SharePoint destination folder.

Common errors and fixes.

- Server relative urls must start with SPWeb.ServerRelativeUrl: Site Address and the path inside GetFolderByServerRelativeUrl don't match. Re-decode the SharePoint URL and align both.
- Folder not found / 404: parent folders must exist. Create them in SharePoint first.
- Access denied: the connection account lacks edit permission on the destination library. Either grant claude@ access, or change the connection on the HTTP action.
- Trigger never fires: confirm Subject Filter is exact (no leading space, includes the colon) and Only with Attachments is Yes.
- File saves empty: the Body field is wrong. Use Attachments Content (per-item, inside the loop), not Attachments (the array).

Cloning for new automations. Don't rebuild from scratch — export the working flow, re-import it under a new name, then change two fields: Subject Filter (trigger) and the destination path inside the Uri. Everything else stays the same.

TYPE

01

SOP

7. Standard shell script shape

Every shell script under bin/ must:

- Source common.sh first (provides PATH and DSS_* path constants).
- Use 'set -uo pipefail' for safer scripts. Do NOT use 'set -e' — handle errors explicitly.
- Use notify-teams for all status (success, failure, info, review).
- Log to \$PLATFORM_LOGS/<automation>/<task>-<date>-<ts>.log.
- Quote all path variables. Paths contain spaces because of SharePoint folder names.
- Handle empty input gracefully — exit cleanly when there is nothing to do.
- For files synced via OneDrive, wait for size stability before reading.
- Move processed inputs to processed/ subfolder to prevent re-runs.

Standard template (file-drop / scheduled triggers):

```
#!/usr/bin/env bash
set -uo pipefail
source "$HOME/automation-platform/platform/lib/common.sh"

LOG_DIR="$PLATFORM_LOGS/<your-automation-name>"
mkdir -p "$LOG_DIR"

# ... your logic ...

notify-teams "Title" "Detail message" success
```

TYPE

01

SOP

7. STANDARD SHELL SCRIPT SHAPE (CONT.)

For email-triggered automations, the script is invoked by the email-watcher with these arguments:

```
#!/usr/bin/env bash
set -uo pipefail
source "$HOME/automation-platform/platform/lib/common.sh"

EMAIL_BODY="$1"      # path to email body (text file)
ATTACHMENTS="$2"     # path to attachments folder (may be empty)
SENDER="$3"          # sender email address
SUBJECT="$4"         # full subject line

# ... process email body / attachments ...

notify-teams "Output ready" "..." review --file <sharepoint-url>
```

8. Available platform helpers

notify-teams — posts an Adaptive Card to the DSS Automation Status or Review channel.

```
notify-teams <title> <message> [level] [--file <sharepoint-url>]
# level: info | success | warning | error | review (default: info)
# review routes the card to the Review channel for human approval
# --file: clickable SharePoint URL, attached to the card
```

File references must be clickable SharePoint links.

When a notification refers to any file — input, output, processed item, attachment, log — the Teams card must include a clickable SharePoint URL. Plain filenames, OneDrive-synced local paths, or text-only mentions are not acceptable. Use the file's `https://...sharepoint.com/...` link, never the `/Users/claude/Library/CloudStorage/...` path.

TYPE

01

SOP

8. AVAILABLE PLATFORM HELPERS (CONT.)

secret — Keychain wrapper. All entries are auto-prefixed with dss-.

```
secret get <name>      # prints value
secret set <name>      # prompts for value, stores
secret delete <name>   # removes
```

Path constants exposed by common.sh:

```
DSS_HUB                ~/dss-hub
DSS_MASTER_DATA        $DSS_HUB/00_Master_Data
DSS_SALES               $DSS_HUB/01_Sales
DSS_INVENTORY           $DSS_HUB/02_Inventory
DSS_PROCUREMENT         $DSS_HUB/03_Procurement
DSS_OPERATIONS_SOPS     $DSS_HUB/04_Operations_SOPs
DSS_MARKETING           $DSS_HUB/05_Marketing
DSS_AI_WORKSPACE        $DSS_HUB/06_AI_Workspace
DSS_TEMPLATES           $DSS_HUB/07_Templates
DSS_ARCHIVE             $DSS_HUB/99_Archive

DSS_INBOX               $DSS_AI_WORKSPACE/Inbox
DSS_REVIEW              $DSS_AI_WORKSPACE/Review
DSS_APPROVED            $DSS_AI_WORKSPACE/Approved
DSS_AI_LOGS             $DSS_AI_WORKSPACE/Logs
DSS_STATUS              $DSS_AI_WORKSPACE/Status
DSS_STATE               $DSS_AI_WORKSPACE/State

PLATFORM_ROOT           $HOME/automation-platform
PLATFORM_LOGS           $PLATFORM_ROOT/runtime/logs
```

TYPE

01

SOP

9. launchd plist conventions

Critical: invoke `/bin/bash` explicitly.

ProgramArguments must start with `/bin/bash`. Do not rely on the script's shebang. macOS TCC needs to identify the binary holding Full Disk Access permission; going through `/usr/bin/env` makes responsible-process tracking ambiguous and the script will be blocked from reading OneDrive folders.

Correct ProgramArguments pattern:

```
<key>ProgramArguments</key>
<array>
  <string>/bin/bash</string>
  <string>/Users/claude/automation-platform/automations/<name>/bin/run.sh</string>
</array>
```

WatchPaths (file-drop trigger). Use the REAL OneDrive path, not the symlinked `~/dss-hub/` path. Symlink resolution can interfere with launchd's file-system event monitoring. The real path is:

```
/Users/claude/Library/CloudStorage/OneDrive-VapeurExpress/
DSS Data Hub - <NN>_<Folder>/...
```

TYPE

01

SOP

9. LAUNCHD PLIST CONVENTIONS (CONT.)

StartCalendarInterval (scheduled trigger):

```
<key>StartCalendarInterval</key>
<dict>
  <key>Weekday</key>
  <integer>2</integer>          <!-- 0=Sun, 1=Mon, ..., 6=Sat -->
  <key>Hour</key>
  <integer>8</integer>
  <key>Minute</key>
  <integer>0</integer>
</dict>
```

Email-triggered automations do NOT ship a launchd plist. The platform's email-watcher daemon owns the plist and dispatches incoming mail to the right handler.

Install commands (file-drop / scheduled only):

```
cp automations/<name>/launchd/com.dss.automation.<name>.plist \
~/Library/LaunchAgents/
launchctl bootout "gui/${id -u}/com.dss.automation.<name>" \
2>/dev/null || true
launchctl bootstrap "gui/${id -u}" \
~/Library/LaunchAgents/com.dss.automation.<name>.plist
```

TYPE

01

SOP

10. Direct IMAP integration (Pattern B, advanced)

Pattern A (Power Automate, section 6) is the default for email triggers. Use this Pattern B only when Power Automate cannot be used — e.g. integrations with non-Microsoft mail systems, or when sub-minute latency is required (Power Automate has a ~30-60 second polling lag).

When trigger type is email and Pattern B is in use, the launchd plist is owned by the email-watcher daemon, not by the automation. The automation provides only the handler script and manifest fields below.

Required manifest fields for email triggers:

- `trigger.subject_prefix` — the case-insensitive subject line prefix (e.g. "SOP:").
- `trigger.from_filter` — a regex of allowed sender addresses, minimum "@digitalsmokesupplies.com".
- `review_required` — true if outputs must be approved before being saved to production folders.

Handler responsibilities:

- Validate sender. If `from_filter` rejects, exit silently with a Teams warning notification including the sender address.
- Process the email body and any attachments from the path provided in `$2`.
- Write outputs to a temporary location, then post a notify-teams card to the Review channel with a SharePoint link if review is required.
- If no review is needed, write directly to the appropriate `DSS_*` folder and post to the Status channel.
- On success, the email-watcher moves the source email to the `processed/` folder automatically. Do not move it yourself.

Email-watcher dispatch logic, in order:

1. Poll IMAP every 60 seconds for new mail in `claudio@digitalsmokesupplies.com`.
2. Match the subject prefix (case-insensitive) against each automation's manifest.
3. Validate the sender against the matching automation's `from_filter`.
4. Save email body to a temp file, save attachments to `$DSS_INBOX/<automation>/<message-id>/`.
5. Invoke the handler with body, attachments folder, sender, subject as arguments.
6. On exit code 0, move the source email to `processed/`. Otherwise leave for retry.

TYPE

01

SOP

11. Review workflow

Any automation producing a file for human consumption routes through the review workflow. Two Teams channels are involved:

- DSS Automation > Status — operational notifications (success, failure, warnings). Read-only by the team.
- DSS Automation > Review — files awaiting human approval. Approvers: David, Juan, plus the relevant department lead.

Review-required outputs (set review_required: true):

- SOPs and policy documents.
- Reports going to external parties.
- Customer-facing communications.
- Anything modifying production master data.

Direct-publish outputs (set review_required: false):

- Internal status reports.
- Meeting notes (already authored by the team in the meeting).
- Backup archives and logs.

Use the review level on notify-teams to route to the Review channel:

```
notify-teams "SOP draft ready for approval" \  
  "Generated from SOP: email from david@" \  
  review \  
  --file https://...sharepoint.com/.../draft.pdf
```

Approvers act on the file directly in SharePoint and reply in the Teams thread to mark approved or rejected. Approved files are moved by the platform to the appropriate DSS_* folder. Rejected files stay in Review with a comment for the author.

TYPE

01

SOP

12. macOS permissions setup

One-time per Mac mini. Required for any automation that reads or writes OneDrive folders:

1. Open System Settings → Privacy & Security → Full Disk Access.
2. Add /bin/bash (use Cmd+Shift+G in the file picker to type the path).
3. Toggle the entry on.

Without this, launchd-spawned scripts get 'Operation not permitted' errors when accessing OneDrive-synced folders. The email-watcher daemon also requires this entry.

13. Authoring an automation via Claude Desktop

Any DSS team member can author an automation with help from Claude Desktop. This process keeps every AI-generated automation aligned to this SOP regardless of who built it.

1. Open a new conversation in Claude Desktop.
2. Attach this SOP PDF as the first message. Add: "Generate a new automation following this SOP. Specs below."
3. Provide the spec: name (kebab-case), one-sentence description, trigger type (file_drop / scheduled / email) with details, inputs, outputs, required secrets, criticality (low/medium/high), and review_required (true/false).
4. Claude generates manifest.yml, bin/run.sh, launchd plist (if applicable), and README.md.
5. Save the output as a folder on your computer.
6. Email the folder (zipped) to claude@digitalsmokesupplies.com with subject AUTOMATION: <automation-name>.
7. The email-watcher routes the submission into the Review channel. David or Juan reviews, then deploys.

TYPE

01

SOP

14. Git workflow

- All automations live in the single automation-platform repository.
- Branch off main for non-trivial changes.
- Commit messages follow the format: <scope>: <what>. Example: meeting-automation: fix /bin/bash invocation.
- The runtime/ directory is gitignored. Never commit logs, state, or hashes.
- Push to GitHub for backup.

15. Testing checklist

Before going live, verify each of the following:

1. Manual run from Terminal: `./bin/run.sh && echo OK`.
2. For file/scheduled triggers: `launchctl kickstart` fires successfully.
3. For email triggers: send a test email matching `subject_prefix` from a `from_filter` address. Verify handler invokes, output appears in Review or Status, source email moves to processed/.
4. Real trigger fires (file drop, scheduled time, or live email).
5. Teams Status or Review channel receives the expected notification.
6. Any file referenced in a Teams notification opens directly when clicked from the card.
7. `runtime/logs/<name>/*.err.log` is clean.
8. Failure scenarios (missing input, broken auth, rejected sender) produce clear error notifications.
9. Idempotency: running twice on the same input does not duplicate work.

TYPE

01

SOP

16. Common pitfalls

- Using `#!/usr/bin/env bash` and relying on it for `launchd` invocation — TCC blocks file access.
- Using `~/dss-hub/` paths in plist `WatchPaths` — may not trigger.
- Not waiting for OneDrive sync stability — reading half-synced files.
- Forgetting to quote variables — fails on paths with spaces.
- Not handling the empty-inbox case — script crashes on first `launchd-empty-trigger`.
- Storing secrets in `.env` files — use Keychain via the `secret` command instead.
- Posting from `osascript` notifications — invisible on a headless Mac mini, use `notify-team`s.
- Posting filenames or local paths to Teams without a clickable SharePoint URL.
- Power Automate: Site Address and the path inside `GetFolderByServerRelativeUrl` don't match — "Server relative urls must start with `SPWeb.ServerRelativeUrl`".
- Power Automate: HTTP action placed outside the Apply to each loop — dynamic content shows the array, not per-item `Name/Content`.
- Power Automate: destination SharePoint folder doesn't exist — the action does not auto-create parent folders.
- Power Automate (Pattern A only): subject filter typo — trigger never fires.
- Email-watcher (Pattern B): `subject_prefix` not matching exactly (typos, missing colon) — handler never fires.
- Email-watcher (Pattern B): missing or overly permissive `from_filter` — anyone can trigger your automation.

17. Where to look when something breaks

- Teams Status channel — should show what failed.
- Power Automate run history (`make.powerautomate.com` → My flows → the flow → 28-day history) — shows every trigger fire and which step failed.
- SharePoint inbox folder — if the file isn't there, the issue is upstream of the mini.
- `runtime/logs/<name>/*.err.log` — recent script errors on the mini.
- `runtime/logs/email-watcher/*.log` — IMAP and dispatch errors (Pattern B only).
- `launchctl list | grep <name>` — agent status. PID 0 means clean exit; non-zero means crashed.
- `launchctl print gui/(id -u)/<label>` — full agent state.
- IMAP auth errors — check Keychain entry: `secret get email-watcher-pass` (Pattern B only).

TYPE

01

SOP

18. Owner contacts

- Platform owner: David — david@digitalsmokesupplies.com.
- Backup owner: Juan.
- Service identity: claud@digitalsmokesupplies.com (also the email trigger inbox).
- Mac mini access: SSH or Screen Sharing (request credentials from platform owner).
- GitHub repository: github.com/DSS-Distro/automation-platform (private).
- Teams channels: DSS Automation > Status, DSS Automation > Review.

19. Non-compliance

Automations that do not follow this SOP risk being silently disabled, blocked by macOS TCC, or producing data inconsistencies that affect downstream systems. Specifically, an automation that:

- Stores secrets outside Keychain will be removed from the platform on detection.
- Posts notifications outside the standard Teams channels will not be monitored and is treated as an unsupported tool.
- Posts file references to Teams without a clickable SharePoint URL will be flagged at code review and must be remediated before re-enabling.
- Writes to data folders without using DSS_* path constants will be rejected at code review.
- Runs without a manifest.yml is considered untracked and will be disabled.
- Email-triggered: missing from_filter is treated as a security incident and disabled immediately.

TYPE

01

SOP

19. NON-COMPLIANCE (CONT.)

Any automation found to be operating without notification visibility, or modifying production data without owner approval, will be unloaded from launchd (or de-registered from the email-watcher) and require a documented review before being re-enabled.

20. Exceptions

Exceptions to this SOP must be documented and approved by the platform owner before deployment. Acceptable grounds for exception include:

- One-off automations for limited-duration projects (under 30 days), provided they still emit Teams notifications and are tracked in a temporary entry.
- External-vendor scripts that cannot be modified to source common.sh — these must be wrapped by a compliant launcher script.
- Experimental automations during platform-engineering work, isolated to a dedicated experimental/ folder and never scheduled in production launchd.

All exceptions must be recorded in the platform README under an 'Exceptions' section with the date, owner, automation name, and intended end date.

Version 2.1 — What changed.

Added section 6: Power Automate input bridge — the recommended pattern for email-triggered automations. Email comes into Outlook, Power Automate drops attachments into a SharePoint inbox folder, OneDrive syncs to the mini, file_drop trigger fires. Pattern B (Python email-watcher daemon, section 10) is reframed as advanced, retained for non-Microsoft mail or sub-minute latency needs. Sections 7–20 are renumbered up by one. v2.0 brought email triggers, the review workflow, and the Claude Desktop authoring process; v2.1 makes the standard input pattern explicit so no team member has to invent it from scratch.